# HossainMohammad

Shahjalal University of Science and Technology


## 1 Geometry

### 1.1 Geometry Primitives

```
#define EPS 1e-12
#define pi 3.14159265358979311599796346\
854418516159057617187 5L
bool feq(const long double& a, const
    long double& b) {
  long double del = fabsl(a - b);
  if (del <= EPS) return true;
  return del <= EPS * max(fabsl(a),
    fabsl(b));
}
```

### 1.2 Point 2D

**Description:** Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

```
template <class T>
int sgn(T x) {
  return (x > 0) - (x < 0);
  // return (x > EPS) - (x < -EPS); //
    for float
}
template <class T>
struct Point {
  typedef Point P;
  T x, y;
  explicit Point(T x = 0, T y = 0) : x(x
    ), y(y) {}
  bool operator<(P p) const { return tie
    (x, y) < tie(p.x, p.y); }
  bool operator==(P p) const { return
    tie(x, y) == tie(p.x, p.y); } //
    Use feq for float
  P operator+(P p) const { return P(x +
    p.x, y + p.y); }
  P operator-(P p) const { return P(x -
    p.x, y - p.y); }
  P operator*(T d) const { return P(x *
    d, y * d); }
  P operator/(T d) const { return P(x /
    d, y / d); }
  T dot(P p) const { return x * p.x + y
    * p.y; }
  T cross(P p) const { return x * p.y -
    y * p.x; }
  T cross(P a, P b) const { return (a -
    *this).cross(b - *this); }
  T dist2() const { return x * x + y * y
    ; }
  T dist2(P p) const { return (*this - p
    ).dist2(); }
  long double dist() const { return sqrt
    ((long double)dist2()); }
  long double dist(P p) const { return
    (*this - p).dist(); }
  // angle to x-axis in interval [-pi,
    pi]
  double angle() const { return atan2(y,
     x); }
  long double angle(P p) const {
    return atan2((long double)this->
    cross(p), (long double)this->dot(p)
    );
  }
  P unit() const { return *this / dist()
    ; }  // makes dist()=1
  P perp() const { return P(-y, x); }
        // rotates +90 degrees
  P normal() const { return perp().unit
    (); }
  // returns point rotated 'a' radians
    ccw around the origin
  P rotate(long double a) const {
    return P(x * cos(a) - y * sin(a), x
    * sin(a) + y * cos(a));
  }
  P project(P p) const { return *this *
    ((double)dot(p) / dist2()); }
  double scalar_project(P p) const {
     return (double)dot(p) / dist(); }
  P reflect(P p) const { return project(
    p) * 2 - p; }
  // Orientation test: >0 for left turn,
     <0 for right turn, 0 if collinear
  T side(P p1, P p2) const { return (p1
    - *this).cross(p2 - *this); }
  friend ostream& operator<<(ostream& os
    , P p) {
    return os << "(" << p.x << "," << p.
    y << ")";
  }
  // Same direction, length r (returns *
    this if degenerate)
  P truncate(double r) const { double k
    = dist(); return !sgn(k) ? *this :
    *this * (r / k); }
};
```

### 1.3 Polar Sort

```
bool half(P p) {
  return p.y < 0 || (p.y == 0 && p.x <
    0);
}
void polar_sort(vector<P>& v, P o = P(0,
     0)) {
  sort(v.begin(), v.end(), [&](P a, P b)
     {
    P u = a - o, w = b - o;
    return make_tuple(half(u), 0.0, u.
     dist2()) < make_tuple(half(w), u.
     cross(w), w.dist2());
  });
}
```

### 1.4 Line Intersection

**Description:**
**Usage:**
```
auto res = lineInter(s1,e1,s2,e2); if
(res.first == 1) cout << "intersection point
at " << res.second << endl;
```

```
template <class P>
pair<int, P> lineInter(P s1, P e1, P s2,
     P e2) {
  auto d = (e1 - s1).cross(e2 - s2);
  if (d == 0)  // if parallel
    return {-(s1.cross(e1, s2) == 0), P
    (0, 0)};
  auto p = s2.cross(e1, e2), q = s2.
    cross(e2, s1);
  return {1, (s1 * p + e1 * q) / d};
}
```

### 1.5 Line Equation

```
array<T, 3> line_equation(const Point<T
    >&b, const Point<T>&c){
 array<T, 3>res;
 res[0] = b.y - c.y, res[1] = -(b.x - c.
    x), res[2] = b.x*c.y - b.y*c.x;
 if(res[0] < 0)res[0] = -res[0], res[1]
    = -res[1], res[2] = -res[2];
 else if(res[0] == 0 and res[1] < 0)res
    [1] = -res[1], res[2] = -res[2]; //
     for double use feq(res[0], 0)
 return res;
}
```

### 1.6 Segment Distance

**Description:**
**Usage:**
```
Point<double> a, b(2,2), p(1,1); bool
onSegment = segDist(a,b,p) < 1e-10;
```

```
typedef Point<double> P;
double segDist(P& s, P& e, P& p) {
  if (s == e) return (p - s).dist();
  auto d = (e - s).dist2(), t = min(d,
    max(.0, (p - s).dot(e - s)));
  return ((p - s) * d - (e - s) * t).
    dist() / d;
}
```

### 1.7 Segment Intersection

**Description:**
**Usage:**
```
vector<P> inter = segInter(s1,e1,s2,e2); if
(sz(inter)==1) cout << "segments intersect at
" << inter[0] << endl;
```

```
template <class P>
vector<P> segInter(P a, P b, P c, P d) {
  auto oa = c.cross(d, a), ob = c.cross(
    d, b), oc = a.cross(b, c),
      od = a.cross(b, d);
  // Checks if intersection is single
    non-endpoint point.
  if (sgn(oa) * sgn(ob) < 0 && sgn(oc) *
     sgn(od) < 0)
    return {(a * ob - b * oa) / (ob - oa
    )};
  set<P> s;
  if (onSegment(c, d, a)) s.insert(a);
  if (onSegment(c, d, b)) s.insert(b);
  if (onSegment(a, b, c)) s.insert(c);
  if (onSegment(a, b, d)) s.insert(d);
  return {all(s)};
}
```

### 1.8 Side Of

**Description:**
**Usage:**
```
bool left = sideOf(p1,p2,q)==1;
```

```
template <class P>
int sideOf(P& s, P& e, P& p) {
  return sgn(s.cross(e, p));
}
template <class P>
int sideOf(const P& s, const P& e, const
     P& p, double eps) {
  auto a = (e - s).cross(p - s);
  double l = (e - s).dist() * eps;
  return (a > l) - (a < -l);
}
```

### 1.9 On Segment

**Description:** Returns true iff p lies on the line segment from s to e.
**Usage:**
```
\texttt {(segDist(s,e,p)<=epsilon)} instead
when using Point<double>.
```

```
template <class P>
bool onSegment(P s, P e, P p) {
  double d = (e - s).dist();
  if (d < 1e-9) return (p - s).dist() <=
     1e-9;
  return abs((e - s).cross(p - s)) / d
    <= 1e-9 && (s - p).dot(e - p) <= 1e
    -9;
}
```

### 1.10 Linear Transformation

**Description:** Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.

```
typedef Point<double> P;
P linearTransformation(const P& p0,
    const P& p1, const P& q0, const P&
    q1,
                       const P& r) {
  P dp = p1 - p0, dq = q1 - q0, num(dp.
    cross(dq), dp.dot(dq));
  return q0 + P((r - p0).cross(num), (r
    - p0).dot(num)) / dp.dist2();
}
```

### 1.11 Circle Intersection

```
typedef Point<double> P;
bool circleInter(P a, P b, double r1,
    double r2, pair<P, P>* out) {
  if (a == b) {
    assert(r1 != r2);
    return false;
  }
```

```
  P vec = b - a;
  double d2 = vec.dist2(), sum = r1 + r2
     , dif = r1 - r2,
         p = (d2 + r1 * r1 - r2 * r2) /
     (d2 * 2), h2 = r1 * r1 - p * p * d2
     ;
  if (sum * sum < d2 || dif * dif > d2)
     return false;
  P mid = a + vec * p, per = vec.perp()
     * sqrt(fmax(0, h2) / d2);
  *out = {mid + per, mid - per};
  return true;
}
```

## 1.12 Circle Polygon Intersection

```
typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.
     dot(q))
double circlePoly(P c, double r, vector<
     P> ps) {
  auto tri = [&](P p, P q) {
    auto r2 = r * r / 2;
    P d = q - p;
    auto a = d.dot(p) / d.dist2(), b = (
     p.dist2() - r * r) / d.dist2();
    auto det = a * a - b;
    if (det <= 0) return arg(p, q) * r2;
    auto s = max(0., -a - sqrt(det)), t
     = min(1., -a + sqrt(det));
    if (t < 0 || 1 <= s) return arg(p, q
     ) * r2;
    P u = p + d * s, v = q + d * (t - 1)
     ;
    return arg(p, u) * r2 + u.cross(v) /
      2 + arg(v, q) * r2;
  };
  auto sum = 0.0;
  rep(i, 0, sz(ps)) sum += tri(ps[i] - c
     , ps[(i + 1) % sz(ps)] - c);
  return sum;
}
```

## 1.13 Circle Line Intersection

```
template <class P>
vector<P> circleLine(P c, double r, P a,
      P b) {
  P ab = b - a, p = a + ab * (c - a).dot
     (ab) / ab.dist2();
  double s = a.cross(b, c), h2 = r * r -
     s * s / ab.dist2();
  if (h2 < 0) return {};
  if (h2 == 0) return {p};
  P h = ab.unit() * sqrt(h2);
  return {p - h, p + h};
}
```

## 1.14 Circle Segment Intersection

```
template <class P>
vector<P> circleSegmentIntersection(P c,
      double r, P a, P b) {
  P ab = b - a;
  double l2 = ab.dist2();
  if (l2 < 1e-12) return (a - c).dist()
     <= r + 1e-9 ? vector<P>{a} : vector
     <P>{};
  P p = a + ab * (c - a).dot(ab) / l2;
  double h2 = r * r - pow((c - a).cross(
     ab), 2) / l2;
  if (h2 < -1e-9) return {};
  P h = ab.unit() * sqrt(max(0.0, h2));
  vector<P> lpts = abs(h2) < 1e-9 ?
     vector<P>{p} : vector<P>{p - h, p +
      h}, res;
  for (auto &pt : lpts) {
    if ((a - pt).dot(b - pt) <= 1e-9)
     res.push_back(pt);
  }
  if (res.empty() && (a - c).dist() <= r
     + 1e-9 && (b - c).dist() <= r + 1e
     -9) {
    res = {a, b};
  }
  return res;
}
```

## 1.15 Circle Tangents

**Description:** Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0. $O(n)$

```
template <class P>
vector<pair<P, P>> tangents(P c1, double
      r1, P c2, double r2) {
  P d = c2 - c1;
  double dr = r1 - r2, d2 = d.dist2(),
     h2 = d2 - dr * dr;
  if (d2 == 0 || h2 < 0) return {};
  vector<pair<P, P>> out;
  for (double sign : {-1, 1}) {
    P v = (d * dr + d.perp() * sqrt(h2)
     * sign) / d2;
    out.push_back({c1 + v * r1, c2 + v *
     r2});
  }
  if (h2 == 0) out.pop_back();
  return out;
}
```

## 1.16 Minimum Enclosing Circle

**Description:** Computes the minimum circle that encloses a set of points. $O(n)$

```
bool inside(const P& p, const P& c,
     double r) {
  return (p - c).dist() <= r + 1e-9;
}
// For generalized, udpate calc with
     gaussian elimination
pair<double, P> calc(const vector<P>& R)
      {
  if (R.empty()) return {0, P()};
  if (R.size() == 1) return {0, R[0]};
  P c = R.size() == 2 ? (R[0] + R[1]) /
     2 : ccCenter(R[0], R[1], R[2]);
  return {(c - R[0]).dist(), c};
}
pair<double, P> welzl(vector<P>& p,
     vector<P>& R, int n, int D) {
  if (!n || R.size() == D + 1) return
     calc(R);
  auto res = welzl(p, R, n - 1, D);
  if (!inside(p[n - 1], res.second, res.
     first)) {
    R.push_back(p[n - 1]);
    res = welzl(p, R, n - 1, D);
    R.pop_back();
  }
  return res;
}
pair<double, P> mec(vector<P>& p, int D
     = 2) {
  shuffle(p.begin(), p.end(), mt19937
     (1337));
  vector<P> R;
  return welzl(p, R, p.size(), D);
}
```

## 1.17 Inside Polygon

**Description:** Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow. $O(n)$

**Usage:**

```
vector<P> v = {P{4,4}, P{1,2}, P{2,1}}; bool
in = inPolygon(v, P{3, 3}, false);
```

```
bool is_point_on_polygon(vector<P> &p, P
     & z) {
    int n = p.size();
    for (int i = 0; i < n; i++) {
      if (onSegment(p[i], p[(i + 1) % n
     ], z)) return 1;
    }
    return 0;
}
int winding_number(vector<P>& p, P& z) {
      // O(n)
  if (is_point_on_polygon(p, z)) return
     1e9;
  int n = p.size(), ans = 0;
  for (int i = 0; i < n; ++i) {
    int j = (i + 1) % n;
    bool below = p[i].y < z.y;
    if (below != (p[j].y < z.y)) {
      auto orient = z.cross(p[j], p[i]);
      if (abs(orient) < 1e-9) return 0;
      if (below == (orient > 0)) ans +=
     below ? 1 : -1;
    }
  }
  return ans;
}
// -1 if strictly inside, 0 if on the
     polygon, 1 if strictly outside
int is_point_in_polygon(vector<P>& p,  P
     & z) { // O(n)
  int k = winding_number(p, z);
  return k == 1e9 ? 0 : k == 0 ? 1 : -1;
}
```

## 1.18 Polygon Area

**Description:** Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

```
template <class T>
T polygonArea2(vector<Point<T>>& v) {
  T a = v.back().cross(v[0]);
  for(int i = 0; i < (int) v.size() - 1;
      ++i)
    a += v[i].cross(v[i + 1]);
  return a;
}
```

## 1.19 Polygon Center

**Description:** Returns the center of mass for a polygon. Time: $O(n)$

```
typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
  P res(0, 0);
  double A = 0;
  for (int i = 0, j = sz(v) - 1; i < sz(
     v); j = i++) {
    res = res + (v[i] + v[j]) * v[j].
     cross(v[i]);
    A += v[j].cross(v[i]);
  }
  return res / A / 3;
}
```

## 1.20 Polygon Cut

```
vector<P> polygonCut(const vector<P>&
     poly, P s, P e) {
  vector<P> res;
  for (int i = 0; i < (int)poly.size();
     ++i) {
    P cur = poly[i], prev = i ? poly[i -
     1] : poly.back();
    auto a = s.cross(e, cur), b = s.
     cross(e, prev);
    if ((a < 0) != (b < 0)) res.
     push_back(cur + (prev - cur) * (a /
      (a - b)));
    if (a < 0) res.push_back(cur);
  }
  return res;
}
```

## 1.21 Polygon Union

```
double rat(P a, P b) { return sgn(b.x) ?
      a.x / b.x : a.y / b.y; }
double polyUnion(vector<vector<P>>& poly
     ) {
  double ret = 0;
  for (int i = 0; i < (int)poly.size();
     ++i) for (int v = 0; v < (int)poly[
     i].size(); ++v) {
    P A = poly[i][v], B = poly[i][(v +
     1) % (int)poly[i].size()];
    vector<pair<double, int>> segs =
     {{0, 0}, {1, 0}};
    for (int j = 0; j < (int)poly.size()
     ; ++j) if (i != j) {
```

```
      for (int u = 0; u < (int)poly[j].
     size(); ++u) {
        P C = poly[j][u], D = poly[j][(u
     + 1) % (int)poly[j].size()];
        int sc = sideOf(A, B, C), sd =
     sideOf(A, B, D);
        if (sc != sd) {
          double sa = C.cross(D, A), sb
     = C.cross(D, B);
          if (min(sc, sd) < 0) segs.
     emplace_back(sa / (sa - sb), sgn(sc
      - sd));
        } else if (!sc && !sd && j < i
     && sgn((B - A).dot(D - C)) > 0) {
          segs.emplace_back(rat(C - A, B
      - A), 1);
          segs.emplace_back(rat(D - A, B
      - A), -1);
        }
      }
    }
    sort(segs.begin(), segs.end());
    for (auto& s : segs) s.first = min(
     max(s.first, 0.0), 1.0);
    double sum = 0;
    int cnt = segs[0].second;
    for (int j = 1; j < (int)segs.size()
     ; ++j) {
      if (!cnt) sum += segs[j].first -
     segs[j - 1].first;
      cnt += segs[j].second;
    }
    ret += A.cross(B) * sum;
  }
  return ret / 2;
}
```

## 1.22 Extreme Vertex

```
// id of the vertex having maximum dot
     product with z
// polygon must be convex
// top - upper right vertex
// for minimum dot product negate z and
     return -p[id].dot(z)
int extreme_vertex(const vector<P>& p,
     const P& z, const int top) { // O(
     log n)
  int n = p.size();
  if (n == 1) return 0;
  auto ans = p[0].dot(z);
  int id = 0;
  if (p[top].dot(z) > ans) ans = p[top].
     dot(z), id = top;
  int l = 1, r = top - 1;
  while (l < r) {
    int mid = (l + r) / 2;
    if (p[mid + 1].dot(z) >= p[mid].dot(
     z)) l = mid + 1;
    else r = mid;
  }
  if (p[l].dot(z) > ans) ans = p[l].dot(
     z), id = l;
  l = top + 1, r = n - 1;
  while (l < r) {
    int mid = (l + r) / 2;
    if (p[(mid + 1) % n].dot(z) >= p[mid
    ].dot(z)) l = mid + 1;
    else r = mid;
  }
  l %= n;
  if (p[l].dot(z) > ans) ans = p[l].dot(
     z), id = l;
  return id;
}
```

## 1.23 Manhattan MST

```
vector<array<int, 3>> manhattanMST(
     vector<P> ps) {
  vector<int> id(ps.size());
  iota(id.begin(), id.end(), 0);
  vector<array<int, 3>> edges;
  for (int k = 0; k < 4; ++k) {
    sort(id.begin(), id.end(),
         [&](int i, int j) { return (ps[
     i] - ps[j]).x < (ps[j] - ps[i]).y;
     });
    map<int, int> sweep;
    for (int i : id) {
      for (auto it = sweep.lower_bound(-
     ps[i].y); it != sweep.end();
           sweep.erase(it++)) {
        int j = it->second;
        P d = ps[i] - ps[j];
        if (d.y > d.x) break;
        edges.push_back({d.y + d.x, i, j
     });
      }
      sweep[-ps[i].y] = i;
    }
    for (P& p : ps)
      if (k & 1)
        p.x = -p.x;
      else
        swap(p.x, p.y);
  }
  return edges;
}
```

## 1.24 Convex Hull

**Description:** Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull. $O(nlogn)$

```
typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
  int n = pts.size();
  if (n <= 1) return pts;
  sort(all(pts));
  vector<P> h(n + 1);
  int s = 0, t = 0;
  for (int it = 2; it--; s = --t,
     reverse(all(pts)))
    for (P p : pts) {
      while (t >= s + 2 && h[t - 2].
     cross(h[t - 1], p) <= 0) t--;
      h[t++] = p;
    }
  return {h.begin(), h.begin() + t - (t
     == 2 && h[0] == h[1])};
}
```

## 1.25 Hull Diameter

**Description:** Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points). $O(n)$

```
typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
  int n = sz(S), j = n < 2 ? 0 : 1;
  pair<ll, array<P, 2>> res({0, {S[0], S
     [0]}});
  rep(i, 0, j) for (;; j = (j + 1) % n)
     {
    res = max(res, {(S[i] - S[j]).dist2
     (), {S[i], S[j]}});
    if ((S[(j + 1) % n] - S[j]).cross(S[
     i + 1] - S[i]) >= 0) break;
  }
  return res.second;
}
```

## 1.26 Point Inside Hull

**Description:** Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included. $O(logN)$

```
typedef Point<ll> P;
bool inHull(vector<P>& l, P p, bool
     strict = true) {
  int n = l.size(), a = 1, b = n - 1, r
     = !strict;
  if (n < 3) return r && onSegment(l[0],
      l.back(), p);
  if (sideOf(l[0], l[a], l[b]) > 0) swap
     (a, b);
  if (sideOf(l[0], l[a], p) >= r ||
     sideOf(l[0], l[b], p) <= -r) return
      false;
  while (abs(a - b) > 1) {
    int c = (a + b) / 2;
    (sideOf(l[0], l[c], p) > 0 ? b : a)
     = c;
  }
  return sgn(l[a].cross(l[b], p)) < r;
}
```

## 1.27 Line Hull Intersection

**Description:** Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon:

- $(-1,-1)$ if no collision,
- $(i,-1)$ if touching the corner $i$,
- $(i,i)$ if along side $(i,i+1)$,
- $(i,j)$ if crossing sides $(i,i+1)$ and $(j,j+1)$.

In the last case, if a corner $i$ is crossed, this is treated as happening on side $(i,i+1)$. The points are returned in the same order as the line hits the polygon. `extrVertex` returns the point of a hull with the max projection onto a line. $O(logN)$

```
#define cmp(i, j) sgn(dir.perp().cross(
     poly[(i) % n] - poly[(j) % n]))
#define extr(i) cmp(i + 1, i) >= 0 &&
     cmp(i, i - 1 + n) < 0
template <class P>
int extrVertex(vector<P>& poly, P dir) {
  int n = sz(poly), lo = 0, hi = n;
  if (extr(0)) return 0;
  while (lo + 1 < hi) {
    int m = (lo + hi) / 2;
    if (extr(m)) return m;
    int ls = cmp(lo + 1, lo), ms = cmp(m
      + 1, m);
    (ls < ms || (ls == ms && ls == cmp(
     lo, m)) ? hi : lo) = m;
  }
  return lo;
}
#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<
     P>& poly) {
  int endA = extrVertex(poly, (a - b).
     perp());
  int endB = extrVertex(poly, (b - a).
     perp());
  if (cmpL(endA) < 0 || cmpL(endB) > 0)
     return {-1, -1};
  array<int, 2> res;
  rep(i, 0, 2) {
    int lo = endB, hi = endA, n = sz(
     poly);
    while ((lo + 1) % n != hi) {
      int m = ((lo + hi + (lo < hi ? 0 :
     n)) / 2) % n;
      (cmpL(m) == cmpL(endB) ? lo : hi)
     = m;
    }
    res[i] = (lo + !cmpL(hi)) % n;
    swap(endA, endB);
  }
  if (res[0] == res[1]) return {res[0],
     -1};
  if (!cmpL(res[0]) && !cmpL(res[1]))
    switch ((res[0] - res[1] + sz(poly)
     + 1) % sz(poly)) {
      case 0:
        return {res[0], res[0]};
      case 2:
        return {res[1], res[1]};
    }
  return res;
}
```

## 1.28 Half Plane Intersection

```
typedef Point<double> P;
using Line = pair<P, P>;
bool bad(Line a, Line b, Line c) {
  auto res = lineInter(a.first, a.second
     , b.first, b.second);
  return res.first > 0 && sideOf(c.first
     , c.second, res.second, EPS) < 0;
}
vector<P> halfPlaneIntersection(vector<
     Line>& L) {
  auto ang = [](Line l) { return (l.
     second-l.first).angle(); };
  sort(all(L), [&](Line a,Line b) {
     return ang(a)<ang(b); });
```

```
  vector<Line> lines;
  for (int i = 0; i < L.size(); i++) {
    if (i && abs(ang(L[i])-ang(L[i-1]))<
     EPS) {
      if (sideOf(L[i].first,L[i].second,
     lines.back().first,EPS) < 0) {
        lines.pop_back(); lines.
     push_back(L[i]);
      }
    } else lines.push_back(L[i]);
  }
  int n = lines.size(), qf = 0, qb = 0;
  vector<Line> q(n);
  for (int i = 0; i < n; i++) {
    while (qb-qf >= 2 && bad(q[qb-1], q[
     qb-2], lines[i])) qb--;
    while (qb-qf >= 2 && bad(q[qf], q[qf
     +1], lines[i])) qf++;
    q[qb++] = lines[i];
  }
  while (qb-qf >= 3 && bad(q[qb-1], q[qb
     -2], q[qf])) qb--;
  while (qb-qf >= 3 && bad(q[qf], q[qf
     +1], q[qb-1])) qf++;
  if(qb-qf < 3) return {};
  vector<P> poly;
  for (int i = qf; i < qb; i++) {
    int nxt = (i+1 == qb) ? qf : i + 1;
    poly.push_back(lineInter(q[i].first,
      q[i].second, q[nxt].first, q[nxt].
     second).second);
  }
  return poly;
}
```

### 1.29 Minkowski Sum Dif

```
using P = Point<ld>;
void reorder(vector<P>& poly) {
  int pos = 0;
  for (int i = 1; i < (int)poly.size();
     ++i)
    if (poly[i] < poly[pos]) pos = i;
  rotate(poly.begin(), poly.begin() +
     pos, poly.end());
}
vector<P> minkowski(vector<P> a, vector<
     P> b) {
  reorder(a), reorder(b);
  int n = a.size(), m = b.size(), ai =
     0, bi = 0;
  a.push_back(a[0]), a.push_back(a[1]);
  b.push_back(b[0]), b.push_back(b[1]);
  vector<P> c;
  while (ai < n || bi < m) {
    c.push_back(a[ai] + b[bi]);
    if (ai < n && bi < m) {
      auto ori = (a[ai + 1] - a[ai]).
     cross(b[bi + 1] - b[bi]);
      if (ori > 1e-9) ai++;
      else if (ori < -1e-9) bi++;
      else ai++, bi++;
    } else if (ai < n) ai++;
    else bi++;
  }
  return c;
}
vector<P> minkowski_dif(vector<P> a,
     vector<P> b) {
  for (auto& p : b) p.x = -p.x, p.y = -p
     .y;
  return minkowski(a, b);
}
```

### 1.30 Maximum Inscribed Circle

```
ld maxInscribedCircle(vector<P>& poly) {
  ld lo = 0, hi = 1e6, ans = 0, L = 1e9;
  int n = poly.size();
  while (hi - lo >= 1e-12) {
    ld mid = lo + (hi - lo) / 2.0;
    vector<Line> l = {{{-L, -L}, {L, -L
     }}, {{L, -L}, {L, L}}, {{L, L}, {-L
     , L}}, {{-L, L}, {-L, -L}}};
    for (int i = 0; i < n; ++i) {
      auto a = poly[i], b = poly[(i + 1)
     % n];
      auto c = (b - a).unit().perp();
      l.push_back({a + c * mid, b + c *
     mid});
    }
    auto cur = halfPlaneIntersection(l);
    if (!cur.empty()) ans = lo = mid;
    else hi = mid;
  }
  return ans;
}
```

### 1.31 Rotating Calipers

```
// Outer loop advances edge (i -> i+1),
     inner loop advances antipodal
     pointer j
for (int i = 0, j = 1; i < n; ++i) {
  while ((p[(i + 1) % n] - p[i]).cross(p
     [(j + 1) % n] - p[j]) > 0) {
    j = (j + 1) % n;
  }
  // Process antipodal pair: p[i] (or
     edge i -> i+1) and p[j]
}
```

### 1.32 Closest Pair

**Description:** Finds the closest pair of points. O(nlogn)

```
typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
  assert(sz(v) > 1);
  set<P> S;
  sort(all(v), [](P a, P b) { return a.y
     < b.y; });
  pair<ll, pair<P, P>> ret{LLONG_MAX, {P
     (), P()}};
  int j = 0;
  for (P p : v) {
    P d{1 + (ll)sqrt(ret.first), 0};
    while (v[j].y <= p.y - d.x) S.erase(
     v[j++]);
    auto lo = S.lower_bound(p - d), hi =
     S.upper_bound(p + d);
    for (; lo != hi; ++lo) ret = min(ret
     , {(*lo - p).dist2(), {*lo, p}});
    S.insert(p);
  }
  return ret.second;
}
```

# 2 Geometry 3D

## 2.1 Point 3D

```
using P2 = Point<double>;
struct P3 {
  double x, y, z;
  explicit P3(double x = 0, double y =
     0, double z = 0) : x(x), y(y), z(z)
     {}
  bool operator<(P3 p) const { return
     tie(x, y, z) < tie(p.x, p.y, p.z);
     }
  bool operator==(P3 p) const { return
     tie(x, y, z) == tie(p.x, p.y, p.z);
     }
  P3 operator+(P3 p) const { return P3(x
     + p.x, y + p.y, z + p.z); }
  P3 operator-(P3 p) const { return P3(x
     - p.x, y - p.y, z - p.z); }
  P3 operator*(double d) const { return
     P3(x * d, y * d, z * d); }
  P3 operator/(double d) const { return
     P3(x / d, y / d, z / d); }
  double dot(P3 p) const { return x * p.
     x + y * p.y + z * p.z; }
  P3 cross(P3 p) const { return P3(y * p
     .z - z * p.y, z * p.x - x * p.z, x
     * p.y - y * p.x); }
  double dist2() const { return x * x +
     y * y + z * z; }
  double dist2(P3 p) const { return (*
     this - p).dist2(); }
  double dist() const { return sqrt(
     dist2()); }
  double dist(P3 p) const { return (*
     this - p).dist(); }
  P3 unit() const { return *this / dist
     (); }
  double angle(P3 p) const { return
     atan2(cross(p).dist(), dot(p)); }
  // (this*a).b
  double triple(P3 a, P3 b) const {
     return cross(a).dot(b); }
  // Smallest angle between this and p,
     ignoring direction, in [0, PI/2]
  double small_angle(P3 p) const {
     return acos(min(fabs(dot(p)) / dist
     () / p.dist(), 1.0)); }
  // Same direction, length r
  P3 truncate(double r) const { double k
     = dist(); return !sgn(k) ? *this :
     *this * (r / k); }
  // Rotate vector around unit axis 'u'
     by angle 'a' (Rodrigues' rotation
     formula)
  P3 rotate(P3 u, double a) const {
    u = u.unit();
    return *this * cos(a) + u.cross(*
     this) * sin(a) + u * u.dot(*this) *
     (1 - cos(a));
  }
  // Returns an arbitrary orthogonal
     unit vector to this
  P3 perp() const { return fabs(x) > EPS
     || fabs(y) > EPS ? P3(-y, x, 0).
     unit() : P3(0, -z, y).unit(); }
};
 // orient' is >0 if s lies on lies on
     the same side of plane pqr as (q-p)
     x (r-p), <0 opposite, 0 on it.
 // orient_by_normal: 2D-like
     orientation of p->q->r on the plane
     with normal n.
double orient(P3 p, P3 q, P3 r, P3 s) {
     return (q - p).cross(r - p).dot(s -
     p); }
double orient_by_normal(P3 p, P3 q, P3 r
     , P3 n) { return (q - p).cross(r -
     p).dot(n); }
```

## 2.2 Plane

```
// 3D Plane: normal n, offset d (n.dot(p
     ) = d)
struct Plane {
  P3 n; double d;
  Plane(P3 n, double d) : n(n), d(d) {}
  Plane(P3 n, P3 p) : n(n), d(n.dot(p))
     {}
  Plane(P3 p, P3 q, P3 r) : Plane((q - p
     ).cross(r - p), p) {}
  double side(P3 p) const { return n.dot
     (p) - d; }
  double dist(P3 p) const { return fabs(
     side(p)) / n.dist(); }
  P3 proj(P3 p) const { return p - n * (
     side(p) / n.dist2()); }
  P3 reflect(P3 p) const { return p - n
     * (2 * side(p) / n.dist2()); }
  // [-PI, PI]
  double angle(Plane p) const { return n
     .angle(p.n); }
  // [0, PI/2]
  double small_angle(Plane p) const {
     return n.small_angle(p.n); }
  bool is_para(Plane p) const { return n
     .cross(p.n).dist() < EPS; }
  bool is_perp(Plane p) const { return
     sgn(n.dot(p.n)) == 0; }
  // Translate the plane by vector t
  Plane translate(P3 t) const { return {
     n, d + n.dot(t)}; }
  // Shift the plane along n by distance
     dis
  Plane shift_up(double dis) const {
     return {n, d + dis * n.dist()}; }
  // Two distinct points lying on the
     plane (n must be non-zero)
  pair<P3, P3> two_pts() const {
    int sx = sgn(n.x), sy = sgn(n.y), sz
     = sgn(n.z);
    assert(sx || sy || sz);
    if (!sx && !sy) return {P3(1, 0, d /
     n.z), P3(0, 1, d / n.z)};
    if (!sy && !sz) return {P3(d / n.x,
     1, 0), P3(d / n.x, 0, 1)};
```

```
    if (!sz && !sx) return {P3(1, d / n.
     y, 0), P3(0, d / n.y, 1)};
    if (!sx) return {P3(1, d / n.y, 0),
     P3(0, 0, d / n.z)};
    if (!sy) return {P3(0, 1, d / n.z),
     P3(d / n.x, 0, 0)};
    if (!sz) return {P3(d / n.x, 0, 1),
     P3(0, d / n.y, 0)};
    if (sgn(d)) return {P3(d / n.x, 0,
     0), P3(0, d / n.y, 0)};
    return {P3(n.y, -n.x, 0), P3(-n.y, n
     .x, 0)};
  }
};
// Intersection line of two planes 3-
    point pairs.
pair<P3, P3> plane_inter(P3 a1, P3 b1,
    P3 c1, P3 a2, P3 b2, P3 c2) {
  auto n1 = (b1 - a1).cross(c1 - a1), n2
     = (b2 - a2).cross(c2 - a2);
  auto d = n1.cross(n2);
  if (d.dist2() == 0) return {P3(), P3()
    };
  auto o = (n2 * n1.dot(a1) - n1 * n2.
    dot(a2)).cross(d) / d.dist2();
  return {o, o + d};
}
```

## 2.3 Line 3D

```
// 3d line parametric form: `o + k * d`.
struct Line3D {
  P3 d, o;
  Line3D() {}
  Line3D(P3 p, P3 q) : d(q - p), o(p) {}
  Line3D(Plane p1, Plane p2) {
    d = p1.n.cross(p2.n);
    o = (p2.n * p1.d - p1.n * p2.d).
     cross(d) / d.dist2();
  }
  double dist2(P3 p) const { return d.
    cross(p - o).dist2() / d.dist2(); }
  double dist(P3 p) const { return sqrt(
    dist2(p)); }
  P3 proj(P3 p) const { return o + d * (
    d.dot(p - o) / d.dist2()); }
  P3 inter(Plane p) const { return o - d
     * (p.side(o) / d.dot(p.n)); }
  // Orthogonal reflection of point p
    across the line
  P3 reflect(P3 p) const { return proj(p
    ) * 2 - p; }
  // Compare points by their projection
    on the line (use as a sort
    comparator)
  bool cmp_proj(P3 p, P3 q) const {
    return d.dot(p) < d.dot(q); }
  // Small angle between the direction
    vectors of two lines
  double angle(Line3D l) const { return
    d.small_angle(l.d); }
  bool is_para(Line3D l) const { return
    d.cross(l.d).dist() < EPS; }
  bool is_perp(Line3D l) const { return
    sgn(d.dot(l.d)) == 0; }
  // Distance from 3D segment [p, q] to
    point x
  static double seg_dist(P3 p, P3 q, P3
    x) {
    if ((q - p).dot(x - p) < 0) return x
    .dist(p);
    if ((p - q).dot(x - q) < 0) return x
    .dist(q);
    return (q - p).unit().cross(x - p).
    dist();
  }
  // Check if point p lies on segment
    defined by [o, o + d]
  bool on_segment(P3 p) const {
    return dist(p) < EPS && (o - p).dot(
    o + d - p) <= EPS;
  }
};
// Line-line relations: min dist,
    closest point
double line_dist(Line3D l1, Line3D l2) {
  auto n = l1.d.cross(l2.d);
  if (n.dist2() == 0) return l1.dist(l2.
    o);
  return fabs((l2.o - l1.o).dot(n)) / n.
    dist();
}
P3 line_closest(Line3D l1, Line3D l2) {
  auto n2 = l2.d.cross(l1.d.cross(l2.d))
    ;
  return l1.o + l1.d * ((l2.o - l1.o).
    dot(n2) / l1.d.dot(n2));
}
```

## 2.4 Plane Line

```
// Plane-line angles, relations, and
    perpendiculars through a point.
double angle(Plane p, Line3D l) { return
     PI / 2 - p.n.small_angle(l.d); }
bool is_para(Plane p, Line3D l) { return
     sgn(p.n.dot(l.d)) == 0; }
bool is_perp(Plane p, Line3D l) { return
     p.n.cross(l.d).dist() < EPS; }
Line3D perp_through(Plane p, P3 o) {
    return Line3D(o, o + p.n); }
Plane perp_through(Line3D l, P3 o) {
    return Plane(l.d, o); }
```

## 2.5 Convex Hull 3d

```
// O(N^2) Incremental 3D Convex Hull
    builder (outputs flattened triangle
    vector)
struct CH3D {
  struct face { int a, b, c; bool ok; };
  int n, num;
  vector<P3> P;
  vector<face> F;
  vector<vector<int>> g;
  void init(vector<P3> p) {
    P = p; n = p.size(); num = 0;
    F.resize(8 * n + 1); g.assign(n + 1,
     vector<int>(n + 1, 0));
  }
  double volume(P3 a, P3 b, P3 c, P3 d)
    { return ((b - a).cross(c - a)).dot
    (d - a); }
  double dblcmp(P3 &p, face &f) { return
     ((P[f.b] - P[f.a]).cross(P[f.c] -
    P[f.a])).dot(p - P[f.a]); }
  void deal(int p, int a, int b) {
    int f = g[a][b]; face add;
    if (F[f].ok) {
      if (dblcmp(P[p], F[f]) > EPS) dfs(
     p, f);
      else {
        add = {b, a, p, true};
        g[p][b] = g[a][p] = g[b][a] =
     num;
        F[num++] = add;
      }
    }
  }
  void dfs(int p, int now) {
    F[now].ok = false;
    deal(p, F[now].b, F[now].a);
    deal(p, F[now].c, F[now].b);
    deal(p, F[now].a, F[now].c);
  }
  void build() {
    if (n < 4) return;
    bool ok = false;
    for (int i = 1; i < n; i++) if (P
     [0].dist(P[i]) > EPS) { swap(P[1],
     P[i]); ok = true; break; }
    if (!ok) return; ok = false;
    for (int i = 2; i < n; i++) if ((P
     [0] - P[1]).cross(P[1] - P[i]).dist
     () > EPS) { swap(P[2], P[i]); ok =
     true; break; }
    if (!ok) return; ok = false;
    for (int i = 3; i < n; i++) if (fabs
     (((P[0] - P[1]).cross(P[1] - P[2]))
     .dot(P[0] - P[i])) > EPS) { swap(P
     [3], P[i]); ok = true; break; }
    if (!ok) return;
    for (int i = 0; i < 4; i++) {
      face add = {(i + 1) % 4, (i + 2) %
     4, (i + 3) % 4, true};
      if (dblcmp(P[i], add) > 0) swap(
     add.b, add.c);
      g[add.a][add.b] = g[add.b][add.c]
     = g[add.c][add.a] = num;
      F[num++] = add;
    }
    for (int i = 4; i < n; i++) {
      for (int j = 0; j < num; j++) if (
     F[j].ok && dblcmp(P[i], F[j]) > EPS
     ) { dfs(i, j); break; }
    }
    int tmp = num; num = 0;
    for (int i = 0; i < tmp; i++) if (F[
     i].ok) F[num++] = F[i];
  }
  // center of mass
  P3 centroid() {
    P3 ans(0, 0, 0), o(0, 0, 0); double
     all = 0;
    for (int i = 0; i < num; i++) {
      double vol = volume(o, P[F[i].a],
     P[F[i].b], P[F[i].c]);
      ans = ans + (o + P[F[i].a] + P[F[i
     ].b] + P[F[i].c]) / 4.0 * vol;
      all += vol;
    }
    return ans / all;
  }
  // Total surface area of the 3D Convex
     Hull
  double area() {
    if (n == 3) return (P[1] - P[0]).
     cross(P[2] - P[0]).dist() * 0.5;
    double res = 0;
    for (int i = 0; i < num; i++) {
      res += (P[F[i].b] - P[F[i].a]).
     cross(P[F[i].c] - P[F[i].a]).dist()
     ;
    }
    return res * 0.5;
  }
  // Total volume enclosed by the 3D
    Convex Hull
  double volume() {
    double res = 0; P3 o(0, 0, 0);
    for (int i = 0; i < num; i++) res +=
      volume(o, P[F[i].a], P[F[i].b], P[
     F[i].c]);
    return fabs(res / 6.0);
  }
  // Are the triangular faces s and t
    coplanar?
  bool same(int s, int t) {
    P3 a = P[F[s].a], b = P[F[s].b], c =
     P[F[s].c];
    return fabs(volume(a, b, c, P[F[t].a
    ])) < EPS && fabs(volume(a, b, c, P
    [F[t].b])) < EPS &&
           fabs(volume(a, b, c, P[F[t].c
    ])) < EPS;
  }
  int num_tri() { return num; } //
    number of surface triangles
  int num_poly() {                //
    number of surface polygons, O(num^2)
    int res = 0;
    for (int i = 0; i < num; i++) {
      bool ok = true;
      for (int j = 0; j < i; j++) if (
     same(i, j)) { ok = false; break; }
      res += ok;
    }
    return res;
  }
  // Distance from point p to the plane
    of face i
  double pt_face_dist(P3 p, int i) {
    return fabs(volume(P[F[i].a], P[F[i
    ].b], P[F[i].c], p)) /
           (P[F[i].b] - P[F[i].a]).cross
    (P[F[i].c] - P[F[i].a]).dist();
  }
  // Check if point p is inside or on
    the 3D Convex Hull
  bool contains(P3 p) {
    for (int i = 0; i < num; i++) {
      if (dblcmp(p, F[i]) > EPS) return
     false;
    }
    return true;
```

```
  }
};
```

## 2.6 Sphere

```
// Spherical coordinates, distances,
    and sphere-sphere intersections.
P3 sph_pt(double r, double lat, double
    lon) {
  lat *= PI / 180; lon *= PI / 180;
  return P3(r * cos(lat) * cos(lon), r *
     cos(lat) * sin(lon), r * sin(lat))
    ;
}
double great_circle_dist(P3 o, double r,
     P3 a, P3 b) { return r * (a - o).
    angle(b - o); }
struct Sphere {
  P3 c; double r;
  double volume() const { return 4.0 /
    3.0 * PI * r * r * r; }
  double surface_area() const { return 4
     * PI * r * r; }
  // Line-Sphere Intersection Points
  vector<P3> inter(Line3D l) const {
    P3 proj = l.proj(c);
    double d2 = c.dist2(proj);
    if (d2 > r * r + EPS) return {};
    double h = sqrt(max(0.0, r * r - d2)
    );
    P3 dir = l.d.unit();
    if (h < EPS) return {proj};
    return {proj - dir * h, proj + dir *
     h};
  }
  // if this point is inside the sphere
  bool contains(P3 p) const { return sgn
    (p.dist(c) - r) <= 0; }
  // Length of the tangent segment from
    an external point p (-1 if p is
    inside)
  double tangent_len(P3 p) const {
    double d = p.dist(c);
    return sgn(d - r) < 0 ? -1 : sqrt(
    max(0.0, d * d - r * r));
  }
  // Polar angle of the cap of tangency
    points from p (-1 if inside, -2 if
    on the surface)
  double tangent_angle(P3 p) const {
    double d = p.dist(c);
    if (sgn(d - r) < 0) return -1;
    if (sgn(d - r) == 0) return -2;
    return acos(r / d);
  }
};
// sphere-sphere intersection
pair<double, double> sphere_inter(Sphere
     s1, Sphere s2) {
  double d = s1.c.dist(s2.c);
  if (d >= s1.r + s2.r) return {0, 0};
  double R = max(s1.r, s2.r), r = min(s1
    .r, s2.r);
  double x = (R * R - r * r + d * d) /
    (2 * d);
  double h1 = R - x, h2 = (R + r - d) -
    h1;
  double vol = PI * h1 * h1 * (3 * R -
    h1) / 3.0 + PI * h2 * h2 * (3 * r -
    h2) / 3.0;
  double area = 2 * PI * R * h1 + 2 * PI
    * r * h2;
  return {area, vol};
}
// Radius of the circle in which two
    intersecting spheres meet
double sphere_inter_radius(Sphere s1,
    Sphere s2) {
  double d = s1.c.dist(s2.c), R = max(s1
    .r, s2.r), r = min(s1.r, s2.r);
  if (sgn(d - R - r) >= 0) return 0;
  double w = d * d - r * r + R * R;
  return sqrt(max(0.0, 4 * d * d * R * R
    - w * w)) / (2 * d);
}
// Sphere through four non-coplanar
    points
Sphere circum_sphere(P3 a, P3 b, P3 c,
    P3 d) {
  assert(sgn(Plane(a, b, c).side(d)) !=
    0);
  Plane u(a - b, (a + b) / 2), v(b - c,
    (b + c) / 2), w(c - d, (c + d) / 2)
    ;
  Line3D l1(u, v), l2(v, w);
  P3 o = line_closest(l1, l2);
  return {o, o.dist(a)};
}
// Center of the circle through three
    non-collinear coplanar points.
P3 circle_center(P3 a, P3 b, P3 c) {
  P3 v1 = b - a, v2 = c - a;
  double v11 = v1.dot(v1), v22 = v2.dot(
    v2), v12 = v1.dot(v2);
  double base = 0.5 / (v11 * v22 - v12 *
    v12);
  return a + v1 * (base * v22 * (v11 -
    v12)) + v2 * (base * v11 * (v22 -
    v12));
}
// approximate min enclosing sphere
Sphere min_sphere(vector<P3> p) {
  P3 c(0, 0, 0); int n = p.size(), pos =
    0;
  for (auto &i : p) c = c + i;
  c = c / n;
  double ratio = 0.1;
  for (int it = 0; it < 100000; it++) {
    pos = 0;
    for (int i = 1; i < n; i++) if (c.
    dist2(p[i]) > c.dist2(p[pos])) pos
    = i;
    c = c + (p[pos] - c) * ratio;
    ratio *= 0.998;
  }
  return {c, c.dist(p[pos])};
}
```

## 2.7 TetraHedron Volume

```
// Tetrahedron (A, B, C, D)
double vol_tetrahedron(p3 a, p3 b, p3 c,
     p3 d) {
  return fabs((b - a).cross(c - a).dot(d
    - a)) / 6.0;
}
```